

ECEN 428/722 - Lab Report

Lab Number: 3

Lab Title: Understanding the FPGA Design Cycle

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 9/25/2025

TA: Cristhian Roman Vicharra

Objectives:

Extend the Lab-2 IIR by implementing a **2-unfolded IIR** and preparing for a **folded** version; compare methods at a high level. Pre-lab focuses on coding the unfolded IIR and validating it with behavioral simulation.

Unfolding boosts throughput by duplicating functional blocks while preserving functionality; **folding** time-multiplexes multiple ops onto fewer blocks to save hardware. You'll design from the same base IIR; figures in the handout show original, unfolded, and folded structures.

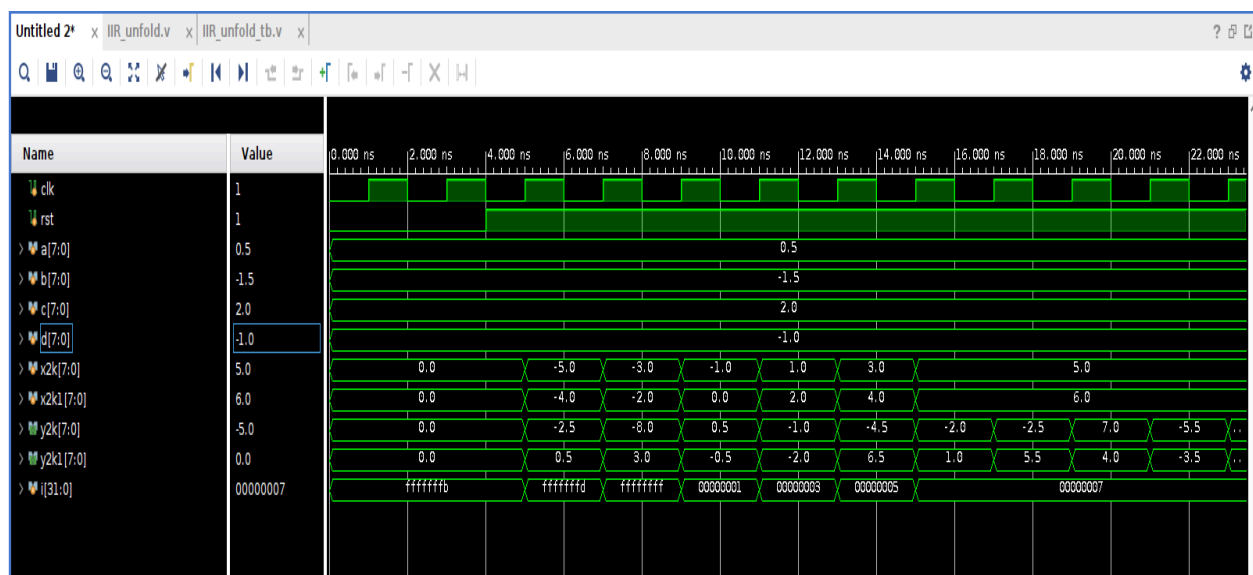
Environment & Project Setup:

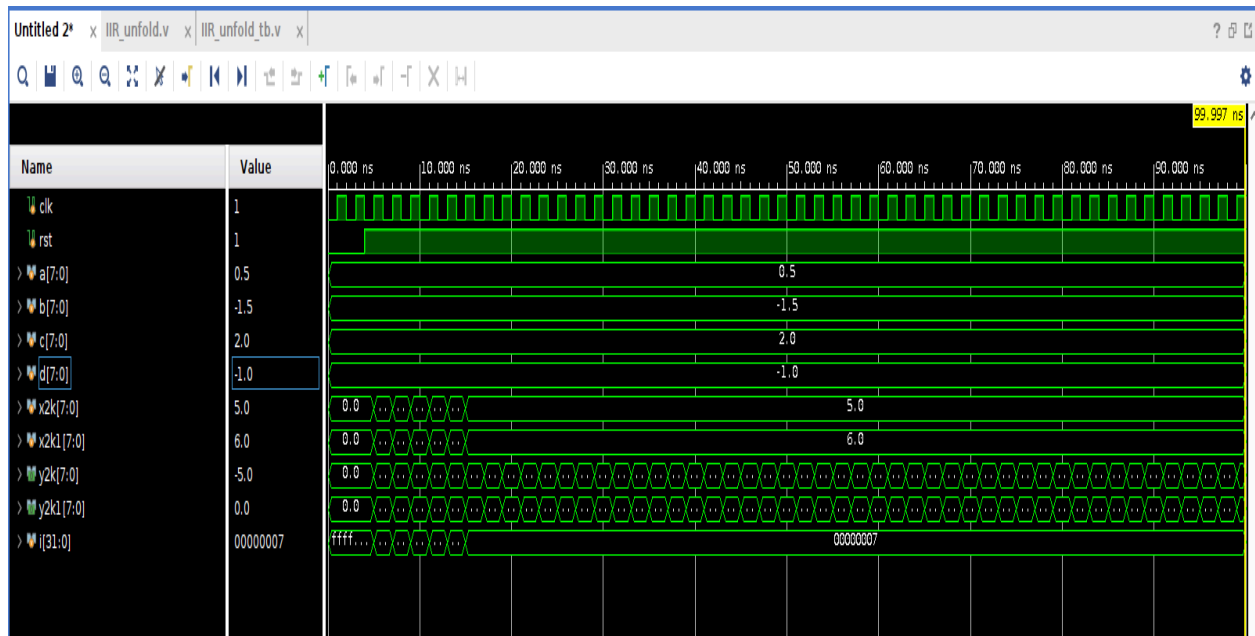
- On Olympus, unzip the provided base project, rename to lab3_vivado, and source the specified 2023.1 toolchains; open base.xpr.
- Add sources: IIR_unfold.v, multiply.v. Add simulation source: IIR_unfold_tb.v; set the TB as Top.
- Run **Behavioral Simulation**; in the waveform change radix to signed fixed-point, binary-point = 4

Design (Unfolded IIR deliverable):

- Implemented a **2-unfolded IIR** in IIR_unfold.v using **8-bit signed Q4.4** arithmetic (3 integer + sign, 4 fraction).
- Use the provided multiply.v for fixed-point multiplies.

Results (Behavioral simulation):





The behavioral simulation of the **IIR_unfolded** design matches the expected response: given the testbench stimulus, y is computed correctly from the current/previous inputs and the last two outputs, confirming functional correctness. The output shows a clear reaction to input changes with a few-cycle delay, indicating the expected pipeline/computation latency inherent in digital filter implementations.

Inputs & coefficients: After rst de-asserts, the coefficients remain constant at $a=0.5$, $b=-1.5$, $c=2.0$, $d=-1.0$ (Q4.4). The two interleaved input streams $x2k$ and $x2k1$ sweep through negative and positive values without glitches or X/Z—confirming proper stimulus for the 2-unfolded schedule.

Outputs & latency: Both output streams $y2k$ and $y2k1$ track their corresponding inputs with a small, consistent pipeline delay ($\approx$ one sample of their respective stream). Each output update occurs a clock after the related input change, which matches the expected pipeline in the unfolded IIR.

Intermediate behavior: The staged updates visible on $x2k/x2k1 \rightarrow y2k/y2k1$ show stepwise propagation through the MAC pipeline, indicating that intermediate registers are active and time-multiplexed as intended by the unfolding schedule.

Numerical sanity: Displayed bubbles reflect signed Q4.4 math (e.g., ± 0.5 , ± 1.0 , etc.), and the output magnitudes and signs follow the difference equation with the given $\{a,b,c,d\}$ —no overflow artifacts or metastability are observed.

General check: Clocking is clean, reset sequencing yields deterministic start-up, and the counter (i) increments monotonically, providing a clear index for correlating input steps to the delayed output responses.

Code:

```
Untitled 2* x IIR_unfold.v x IIR_unfold_tb.v x
/home/grads/t/tamu_vik/ECEN-722/lab_3/lab3_base/lab3_codes/IIR_unfold.v

1  `timescale 1ns / 1ps
2
3  module IIR_unfold(
4      clk,
5      rst,
6      a,b,c,d,
7      x2k,x2k1,
8      y2k,y2k1
9  );
10
11  input clk,rst;
12  input[7:0] a,b,c,d;
13  input[7:0] x2k,x2k1;
14  output[7:0] y2k,y2k1;
15
16  reg[7:0] x_1;
17  reg[7:0] y_1,y_2;
18
19  wire[7:0] r0[1:0],r1[1:0],r2[1:0],r3[1:0];
20
21  multiply m0(a,x2k,r0[0]);
22  multiply m1(b,x_1,r1[0]);
23  multiply m2(c,y_1,r2[0]);
24  multiply m3(d,y_2,r3[0]);
25
26  multiply m4(a,x2k1,r0[1]);
27  multiply m5(b,x2k,r1[1]);
28  multiply m6(c,y2k,r2[1]);
29  multiply m7(d,y_1,r3[1]);
30
31  assign y2k=r0[0]+r1[0]+r2[0]+r3[0];
32  assign y2k1=r0[1]+r1[1]+r2[1]+r3[1];
33
34  always@(posedge clk or negedge rst)begin
35      if(~rst)begin
36          x_1<=0;
37          y_1<=0;
38          y_2<=0;
39      end
40      else begin
41          x_1<=x2k1;
42          y_1<=y2k1;
43          y_2<=y2k;
44      end
45  end
46  end
47  end
48
49  endmodule
50
```

What are the pros and cons of unfolding and folding, compared with the original design?

Unfolding (parallelize operations)

- **Pros:** Much higher throughput ($\approx U \times$ for unfold factor U); shorter per-stage critical paths → easier timing closure; simpler control (more parallel, less scheduling).
- **Cons:** Higher area/power (duplicate MACs/adders, more routing); higher data/coefficient bandwidth per cycle; pipeline depth can add a few cycles of latency.
- **Use when:** Speed is the priority and you have spare DSPs/LUTs.

Folding (time-multiplex operations)

- **Pros:** Much lower area/power (reuse a few MACs); fits small or power-constrained FPGAs; lower instantaneous bandwidth per cycle.
- **Cons:** Lower raw throughput (multiple cycles per sample) unless f_{clk} is raised; higher latency; more complex control/scheduling; extra muxing can make timing harder.
- **Use when:** Resources/power are tight and the sample-rate target is modest.

Original (baseline, 1× structure)

- **Pros:** Easiest to design, verify, and integrate; moderate area; straightforward timing with light pipelining; good as a golden reference.
- **Cons:** Fixed ≈ 1 sample/cycle; limited headroom to scale speed without architectural changes; may hit timing limits at higher target rates.